



Hoja de trabajo 5:

Orquestación

Estudiante: Haroldo Alejandro Flores Argueta

Correro: flo241109@uvg.edu.gt

Carne: 241109

Introducción

En esta hoja de trabajo se implementó una solución para Parachute S.A. utilizando sistemas multiagente con tres formas distintas de orquestación: centralizada, jerárquica y descentralizada. El objetivo principal fue comparar como cambia la organización y comunicación entre los agentes al resolver un mismo conjunto de requerimientos.

El sistema desarrollado conserva la base de conocimientos de preguntas frecuentes implementadas previamente con PostgreSQL y pgvector, e incorpora nuevas funcionalidades para consultar información meteorológica mediante Open-Meteo, evaluar las condiciones de seguridad para realizar un salto y gestionar reservaciones. Para evitar duplicar lógica entre las tres implementaciones, las integraciones y reglas principales se desarrollaron como servicios compartidos, mientras que cada programa modifica principalmente la forma en que los agentes coordinan sus tareas.

Para la implementación de los agentes se utilizó OpenAI Agents SDK junto con proveedores de modelos compatibles con la API de OpenAI. En las arquitecturas centralizada y jerárquica se utilizaron agentes especializados como herramientas mediante `as_tool()`, mientras que en la arquitectura descentralizada se utilizaron handoffs para permitir la transferencia directa del control entre agentes.

Además, las decisiones críticas relacionadas con la seguridad meteorológica fueron implementadas mediante código determinista en lugar de depender exclusivamente del modelo de lenguaje. De esta manera, independientemente de la arquitectura utilizada, se mantienen las mismas reglas para determinar si las condiciones permiten o impiden realizar y calendarizar un salto.

Preguntas por responder.

1. ¿Qué arquitectura/arquitecturas resuelven mejor este problema? ¿Por qué?

Para este problema, considero que la **arquitectura centralizada** es la alternativa más adecuada actualmente. La razón principal es que Parachute S.A. todavía tiene un conjunto relativamente pequeño y claramente definido de responsabilidades: responder preguntas frecuentes, consultar condiciones meteorológicas, evaluar la seguridad de los saltos y gestionar reservaciones.

En la arquitectura centralizada, un Manager Agent recibe la solicitud del usuario y decide qué agente especializado debe utilizar. Los agentes de FAQs, clima y reservaciones se encuentran disponibles para el Manager mediante `as_tool()`. Esto mantiene un único punto de coordinación y hace sencillo controlar el flujo que debe seguir una reservación.

La **arquitectura jerárquica** también resuelve correctamente el problema y podría convertirse en una mejor alternativa si Parachute S.A. continúa aumentando considerablemente sus requerimientos. En nuestra implementación existe un **Root Manager**, debajo del cual se encuentran supervisores especializados. Por ejemplo, el **Booking Supervisor** coordina los agentes relacionados con clima y calendarización. Esta separación permite que nuevos departamentos y agentes puedan agregarse sin concentrar todas las responsabilidades en un único Manager. Sin embargo, para el tamaño actual del sistema introduce niveles adicionales de coordinación que todavía no son indispensables.

La **arquitectura descentralizada** permite que los agentes transfieran directamente el control utilizando `. Esto reduce la dependencia de un supervisor central y permite interacciones más autónomas entre especialistas. Sin embargo, también aumenta la complejidad para controlar procesos que deben seguir un orden estricto. En este problema, una reservación debe validar primero la fecha, consultar el clima, evaluar las condiciones y solamente después crear la cita. Por esta razón, mantener el control de este proceso resulta más sencillo con una arquitectura centralizada.

Un aspecto importante identificado durante la implementación fue que las **reglas críticas de seguridad no deberían depender exclusivamente de las decisiones de un LLM**. Por ello, los límites de viento, ráfagas, precipitación y cobertura de nubes se implementaron mediante código determinista compartido. Además, antes de almacenar una cita se vuelve a verificar la seguridad meteorológica. De esta forma, las tres arquitecturas pueden cambiar su mecanismo de orquestación sin modificar las reglas principales del negocio.

Por lo tanto, para los requerimientos actuales utilizaría la **arquitectura centralizada**, mientras que la arquitectura jerárquica presenta ventajas de organización si el sistema continúa creciendo considerablemente.

2. ¿Considera que es necesario utilizar un sistema multiagente en este caso? ¿Por qué?

No considero que un sistema multiagente sea **estrictamente necesario** para resolver los requerimientos actuales.

El sistema podría implementarse utilizando un único agente equipado con herramientas para consultar la base de FAQs, Open-Meteo y el servicio de calendarización. Las reglas de seguridad tampoco requieren razonamiento de múltiples agentes, ya que pueden y deben evaluarse mediante funciones deterministas.

Sin embargo, utilizar un sistema multiagente sí ofrece ventajas de **separación de responsabilidades y extensibilidad**. Cada agente puede especializarse en un dominio concreto, mientras que las integraciones se mantienen independientes dentro de servicios compartidos. Esta organización resulta especialmente relevante porque Parachute S.A. indicó que continuará incrementando sus requerimientos funcionales.

Por ejemplo, en el futuro podrían agregarse agentes especializados en pagos, disponibilidad de instructores, mantenimiento de equipo, transporte, atención al cliente o gestión de eventos. En ese escenario, separar responsabilidades entre agentes podría facilitar el mantenimiento y crecimiento del sistema.

La implementación de las tres arquitecturas también permitió observar que **agregar más agentes no implica automáticamente obtener una mejor solución**. Cada nivel adicional de coordinación introduce nuevas llamadas al modelo, mayor latencia y más puntos en los que debe controlarse el flujo. Por eso, la cantidad de agentes debería justificarse por la complejidad del dominio y no únicamente por la posibilidad técnica de utilizarlos.

En conclusión, un MAS no es indispensable para la versión actual de Parachute S.A., pero puede justificarse por la necesidad futura de **modularidad, especialización y crecimiento del sistema**. Para el alcance actual, una organización multiagente relativamente simple resulta suficiente.